



UNIVERSIDAD NACIONAL AUTONOMA DE MÉXICO

*Facultad De Estudios Superiores
Aragón*

Documentación Parcial Uno

Alumnos: Barrera Jiménez Christian

Arrieta Flores Alan Alejandro

Pérez Hernández Obet

Materia: Compiladores

Profesor: Miguel Ángel Sánchez Hernández

Grupo: 2609

Fecha: 16/04/26



ÍNDICE

Parcial 1.....	1
Introducción.....	3
Objetivos.....	4
Quicksort.....	10
Conclusión.....	11
Bibliografía.....	13

Introducción

En computación podemos definir un compilador como el intermediario entre el usuario y la máquina, siendo lo que necesitamos para comprender el uno del otro. Un compilador se define con tres fases: analizador léxico, sintáctico y semántico. Estas fases se complementan con la anterior, es decir, si cometemos un error en una fase previa, nuestro compilador se vuelve completamente inservible.

En este parcial, indagamos en específico con el analizador léxico, conociendo así sus elementos y la relación que existe entre ellos.

Comencemos hablando de los lexemas. Un lexema se define como una secuencia de caracteres que pueden ser válidos después de evaluarse con tokens conocidos. Un token, entonces, se vuelven esos bloques (identificadores) que tienen significado y que son representados con un valor e identificador.

Pero entonces ¿cómo podemos definir a estos tokens y lexemas?. Utilizando una gramática de lenguaje o alfabeto, que ya se definió previamente. En esta gramática, vamos a encontrar nuestras palabras reservadas para nuestro computador, es decir, esas operaciones que necesita para poder evaluar los tokens que vamos a validar o invalidar.

Un paso crucial para definir a los tokens, es el uso de expresiones regulares, esto para simplificar al AFD que ocuparemos para obtener ese estado final que nos definirá que tipo de token tenemos.

Al final, este analizador léxico, nos va a permitir categorizar errores o modificaciones y aciertos en nuestros tokens. Aunque también si tuvimos una configuración errónea al clasificar nuestros tokens, prácticamente volvemos a este analizador inservible para su uso posterior.

Objetivos

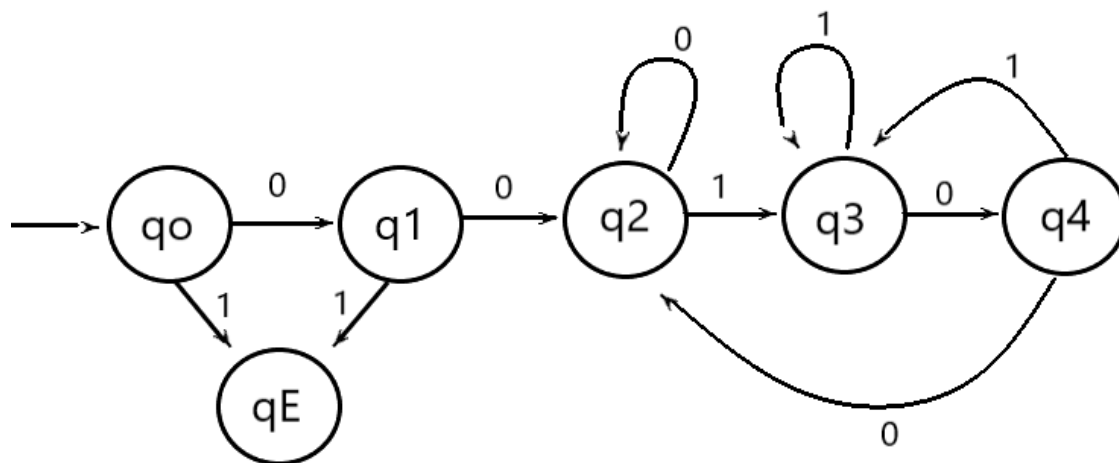
1. Demostrar la integración de un flujo de entrada de datos externo mediante archivos de texto y diálogos de usuario para la alimentación de procesos de análisis léxico, asegurando la correcta interpretación de símbolos y el cierre adecuado de los flujos de lectura.
2. Implementar y configurar un analizador léxico funcional que utilice un flujo de lectura de archivos para la extracción y clasificación de tokens a partir de un código fuente
3. Evaluar el manejo de errores y la recuperación de estados en el analizador léxico, identificando visualmente mediante reportes de consola la precisión en la localización de errores (línea y columna) frente a entradas inválidas.
4. Identificar el funcionamiento de Jflex y utilizarlo para generar un analizador léxico.

1.- Autómata 00^*00 con alfabeto 01

Se necesita un algoritmo que de válidas las cadenas 00^*00 .

En este programa utilizamos un AFD para poder aceptar cadenas de números aleatorios pero que inicien con 00 y terminan con 00.

La clave para el desarrollo de este AFD fue el supuesto caso de tener la entrada "00", porque, es la respuesta que cumple con la mínima cantidad de números con las que cumplimos el estado final. Entonces, ahora solo necesitamos devolver las siguientes entradas a este estado para que puedan ser válidas las cadenas.



Utilizamos el siguiente pseudocódigo:

```
maxSuma(cadena, n) {  
    estado ← 0 i ← 1  
  
    while i ≤ n and estado ≠ -1 do  
        c ← cadena[i]  
  
        switch(estado)  
            case 0: >>>> if (c='0') then estado ← 1  
                        else estado ← -1  
  
            case 1:
```

```
if (c='0') then estado←2 else estado←-1
```

case 2:

```
if (c='0') then estado←2 else if (c='1')  
then estado←3 else estado←-1
```

case 3:

```
if (c='0') then estado←4 else if (c='1')  
then estado←3 else estado←-1
```

case 4:

```
if (c='0') then estado←2 else if (c='1')  
then estado←3 else estado←-1
```

```
i←i+1
```

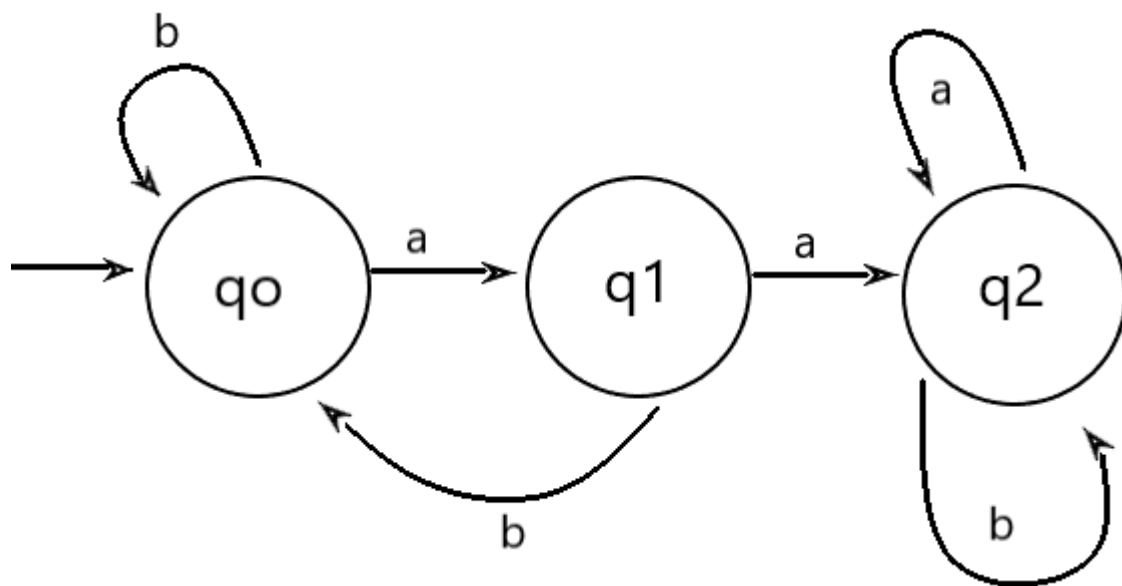
```
if (estado=2) return "Válida" else return "Inválida" }
```

Consideramos que el algoritmo utilizado para representar a este AFD es bastante preciso, ya que nos permite evaluar cuando una cadena termina con un while y posteriormente solo evaluar si el estado final es igual a 2, si es así, la cadena es válida.

2.- Autómata *aa* con el alfabeto ab

El problema es validar solo cadenas que contengan dos "a" juntas, sin importar el orden de los demás caracteres. De nuevo, la pista fue empezar con el supuesto de la cadena "aa", que es la que cumple con los datos mínimos para ser válida. Posteriormente, solo mandamos a los siguientes caracteres al mismo estado, ya que el propósito ya fue atendido.

Este programa tiene una problemática similar al anterior. Lo resolvimos haciendo un AFD que es el siguiente.



El pseudocódigo utilizado fue:

```

validarCadena(cadena, n) {
    estado ← 0 i ← 1

    if (n < 2) return "Inválida"

    while (i ≤ n and estado = -1) do
        c ← cadena[i]

        switch(estado)

            case 0: >>> if (c = 'a') then estado ← 1

                    else if (c = 'b') then estado ← 0 else
                        estado ← -1

            case 1:

                    if (c = 'a') then estado ← 2 else if (c = 'b')
                        then estado ← 0 else estado ← -1

            case 2:

                    if (c = 'a' or c = 'b') then estado ← 2 else
                        estado ← -1

        i ← i + 1

    if (estado = 2) return "Válida" else return "Inválida" }
  
```

No notamos muchas diferencias entre este programa y el anterior, más allá de que se basan en dos AFD distintos.

3.- Identificadores

El problema busca diseñar y programar un diseñador léxico para validar un identificador, estableciendo como regla que este debe comenzar con una letra para posteriormente admitir una secuencia opcional de letras o dígitos. Para modelar esta validación, se emplea un Autómata Finito Determinista (AFD) compuesto por tres estados.

Para el desarrollo de este problema hicimos uso del ciclo while para recorrer la cadena y una estructura anidada de case e if/else para simular las transiciones de estados. Dentro de este flujo, el analizador evalúa el tipo de símbolo leído en función del estado actual.

```
Estado:= 1;
Leer siguiente símbolo
While no fin-cadena do
    case Estado of
        1: if símbolo == letra then Estado:= 3
            else if símbolo == dígito then Estado:= 2
            else
                rutina error
        2: rutina error
        3: if símbolo == letra then Estado := 3
            else if símbolo == dígito then Estado:= 3
            else
                rutina error
    leer el siguiente carácter
end while;
if Estado != 3 then rutina error
else identificador valor
```

El método usando el ciclo do while nos permite ver que usando lógica condicional directa se puede realizar el reconocimiento de un identificador básico pero al mismo tiempo expone las limitaciones de programar por fuerza bruta cada transición del diagrama.

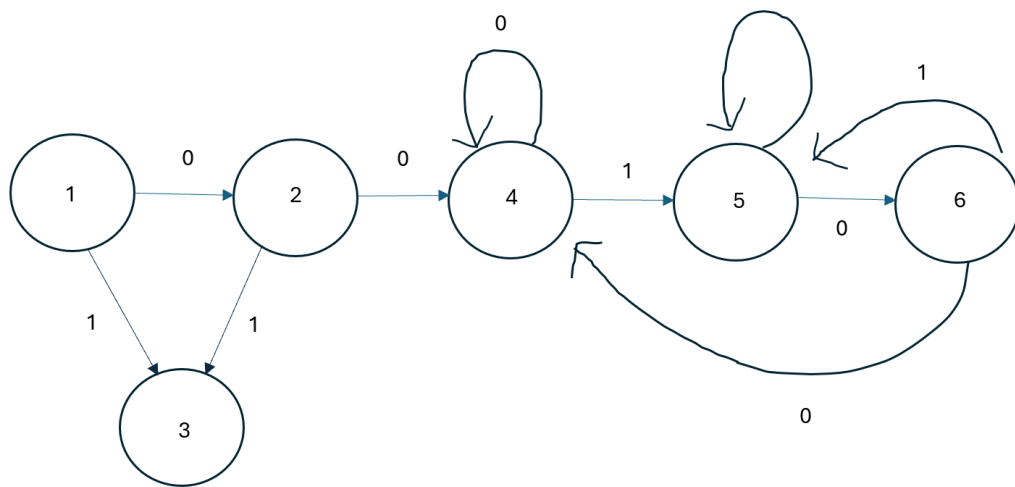
4.- Versión 3 de la Tabla de Transición crear el AFD 1

El programa debe resolver el AFD 00*00 que se presenta en el programa , utilizando una tabla de transición para su desarrollo.

La diferencia entre este programa en el que se ocupa tabla de transición, con el programa 1, es el uso e importancia de la tabla de transición ¿Y por qué ocuparemos una tabla de transición? Bueno, si estuviésemos realizando un AFD con muchos estados de transición e

intentaríamos programar un AFD de forma fuerza bruta, terminaríamos con sentencias gigantes de switch o if/else anidadas para cada estado y cada posible carácter leído del flujo de entrada. Eso es propenso a errores y muy difícil de mantener.

La implementación de una tabla de transición es fundamental ya que permite que el motor de evaluación sea independiente de las reglas del lenguaje, haciendo que sea más sencillo de modificar el código para el AFD, además de esto la integración de una tabla de transición facilita la lectura y depuración del analizador léxico. Además, al sustituir la evaluación secuencial de expresiones booleanas por el acceso directo a una coordenada en la matriz en memoria permitiendo un rendimiento mayor.



Índice / Estado	0	1	FC (Fin de Cadena)
1	2	3	Error
2	4	3	Error
3	3	3	Error
4	4	5	Aceptar
5	6	5	Error
6	4	5	Error

```

Estado:= 1;
repeat
  Leer el siguiente símbolo
  case símbolo of
    0: Entrada:= "0";
    1: Entrada:= "1";
    FC: Entrada:=FC;
    ninguno: rutina error
  Estado :=Tabla[Estado,Entrada]

```



```
if Estado == "error then rutina error
until Estado != "Aceptar"
```

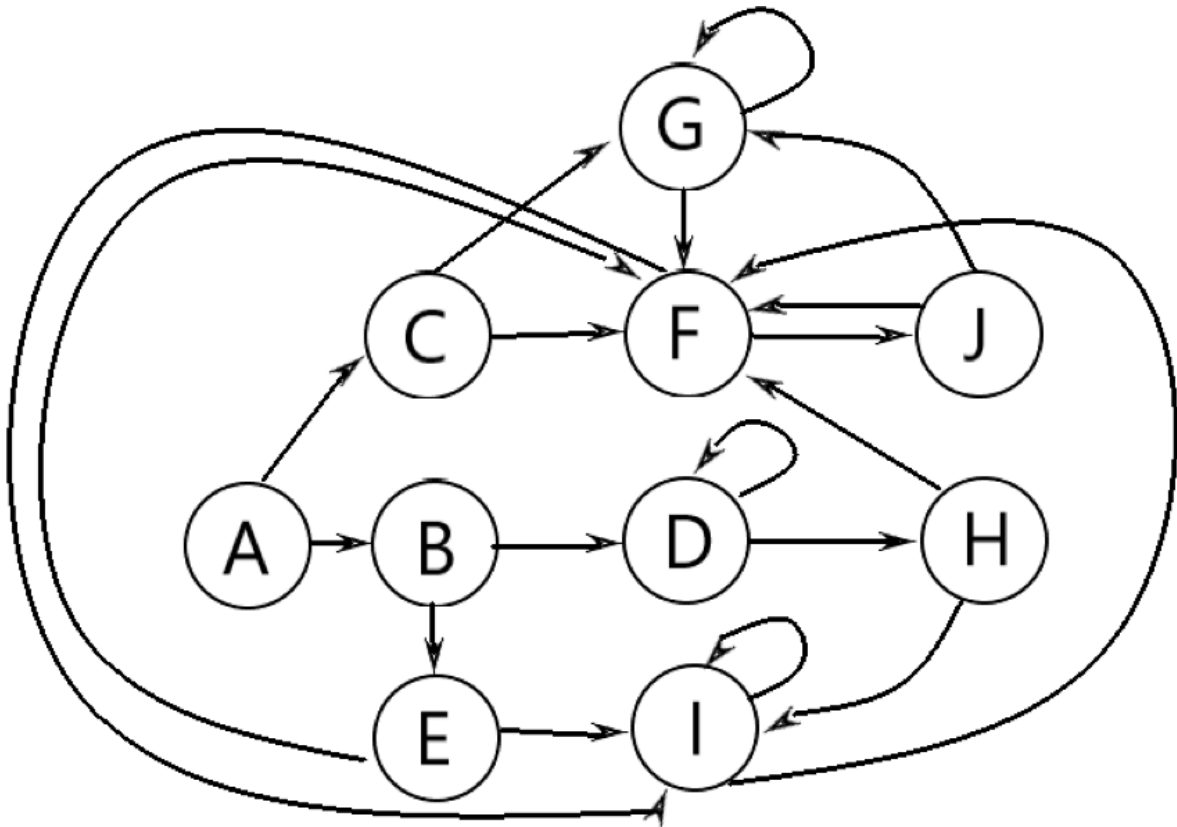
En definitiva, la decisión de implementar el autómata para la expresión 00^*00 mediante una tabla de transición en nuestro archivo va más allá de la simple optimización del rendimiento en tiempo de ejecución. Tomamos esta decisión porque esta estructura nos permite una visualización y un control mucho más preciso de los fallos. Al mapear cada posible cruce entre el estado actual y el símbolo de entrada, los caminos que conducen a un estado de rechazo quedan definidos en la matriz.

5.- Código versión 2 para expresión regular $(0^*1|1^*)01$

En este programa debíamos de utilizar expresiones regulares e implementarlas en una tabla de transición similar al programa anterior.

Podríamos decir que este programa es un caso especial a comparación de los anteriores, ya que reducimos mucha de la memoria que se necesita utilizar para hacer funcionar a el analizador léxico.

En nuestra programación nos basamos casi en su totalidad en el programa anterior, ya que utilizamos los estados para poder optimizar el uso de este pequeño analizador léxico. Primero declaramos el alfabeto que vamos a ocupar y posteriormente con el siguiente AFD declaramos los estados que vamos a utilizar así como sus rutas para llegar al estado final.



Entonces, la tabla de transiciones quedó de la siguiente manera:

ESTADO	0	1	EOF
A	B	C	Error
B	D	E	Error
C	F	G	Error
D	D	H	Error
E	I	F	Error
F	I	J	Error
G	F	G	Error
H	F	I	Error
I	F	I	Error
J	F	G	Aceptado

El pseudocódigo utilizado es:

```

main(args) {
    cadena←leerEntrada("Dame la cadena") indice←1
    if (cadena=nulo) return
    valor←estado_A()
    if (valor=1) then

```

```

        print "Cadena Válida" else print "Cadena Inválida" }

siguienteCaracter() {

    if (indice ≤ longitud de cadena) then

        c ← cadena[indice] indice ← indice + 1 return c return '\0' }

estado_A() {

    c ← siguienteCaracter() switch(c)

        case '0': return estado_B() case '1': return estado_C() default:
        return -1 }

```

(... Los estados B al I siguen la misma lógica de transición según el carácter '0' o '1' ...)

```

estado_J() {

    c ← siguienteCaracter() switch(c)

        case '0': return estado_F() case '1': return estado_G() case '\0':
        return 1 default: return -1 }

```

Para finalizar el programa, de nuevo, evaluamos si el estado final es igual al último estado donde la computadora se quedó al leer la entrada.

Esta forma de trabajar un analizador léxico, nos pareció bastante eficaz para simplificar una gran cantidad de estados y caminos que se pueden tomar.

6.- Autómata de notación científica

El programa es un analizador léxico con interfaz gráfica JavaFX que lee un archivo .txt, separa su contenido en tokens por espacios y determina, para cada token, si corresponde a una expresión en notación científica válida. El núcleo del analizador es un Autómata Finito Determinista (AFD) implementado mediante una tabla de transiciones bidimensional, lo cual permite reducir el uso de memoria a únicamente el estado actual y el índice del carácter en proceso.

La interfaz presenta al usuario la tabla de transiciones del AFD, un área de texto para entrada manual o carga de archivo, y una tabla de resultados que muestra por cada token: el recorrido completo de estados, el tipo de token y si fue aceptado o rechazado

Definición formal — $M = (Q, \Sigma, \delta, q_1, F)$

- $Q = \{ q_1, q_2, q_3, q_4, q_5, q_6, q_7 \}$
- $\Sigma = \{ \text{dígito}, '.', 'e/E', '+/-', \text{otro} \}$
- δ : función de transición definida en la tabla
- q_1 = estado inicial
- $F = \{ q_7 \}$ = único estado de aceptación

Estado	dígito	.	e/E	+/-	otro	¿Aceptación?
q1	q2	ERR	ERR	q2*	ERR	No
q2	q2	q3	q5	ERR	ERR	No
q3	q4	ERR	ERR	ERR	ERR	No
q4	q4	ERR	q5	ERR	ERR	No
q5	q7	ERR	ERR	q6	ERR	No
q6	q7	ERR	ERR	ERR	ERR	No
q7	q7	ERR	ERR	ERR	ERR	Sí

- q1 → Estado inicial. Espera el primer carácter: dígito o signo opcional.
- q2 → Leyendo la parte entera. Acepta más dígitos, punto decimal o 'e/E'.
- q3 → Se leyó el punto decimal. Obligatorio al menos un dígito decimal.
- q4 → Leyendo la parte decimal. Acepta más dígitos o 'e/E'.
- q5 → Se leyó 'e' o 'E'. Espera signo opcional o dígito del exponente.
- q6 → Se leyó el signo del exponente. Obligatorio al menos un dígito.
- q7 → Leyendo el exponente. Estado de aceptación. Acepta más dígitos.

Pseudocódigo:

La lógica del programa replica el esquema de funciones de estado del programa anterior. Cada estado del AFD corresponde a una función que lee el siguiente carácter y aplica la transición correspondiente:

```

main(args) {
    tokens ← leerArchivo(.txt).split(espacios)
    para cada token {
        indice ← 0
        valor ← estado_q1()
        si (valor = 1) imprimir 'Válida'
        sino          imprimir 'Inválida'
    }
}

siguienteCaracter() {
    si (indice < longitud(token)) retornar token[indice++]
    retornar '\0'
}

```

```

estado_q1() {
  c ← siguienteCaracter()
  según (c)
    caso dígito: retornar estado_q2()
    caso '+' '-' : retornar estado_q2()
    default:      retornar -1
}

```

```

estado_q2() {
  c ← siguienteCaracter()
  según (c)
    caso dígito: retornar estado_q2()
    caso '.': retornar estado_q3()
    caso 'e'/'E': retornar estado_q5()
    caso '\0': retornar -1
    default:      retornar -1
}

```

```

estado_q7() {
  c ← siguienteCaracter()
  según (c)
    caso dígito: retornar estado_q7()
    caso '\0': retornar 1
    default:      retornar -1
}

```

Al terminar de procesar cada token se evalúa si el valor retornado es igual a 1. Si lo es, el token llegó al estado de aceptación q7; de lo contrario fue rechazado en alguna transición ERR.

Conclusión: Este programa resultó ser demasiado eficiente en el uso de memoria y bastante escalable ya que utilizamos un AFD para crear tablas de transición, es eficiente en memoria porque se mantiene siempre en el estado actual y consulta sus movimientos en la tabla por lo que solo modificando la tabla puede ser fácilmente escalable y se podría adaptar cualquier autómatas lo complicado sería crear la tabla de transiciones ya que un solo error en los cambios de estado resultará en un error fatal porque nunca llegará a un estado de aceptación y puede generar bucles infinitos, que siempre esté en estado de error y dar falsos positivos por lo demás podemos decir que es el programa más completo para analizadores léxicos.

7.- Versión 3 auto-configurable

El programa era resolver el problema 3 con Jflex.

Jflex es un generador de analizadores léxicos, necesita un código entrada (en este caso el archivo codigo.txt) en el que ponemos las reglas léxicas y las expresiones regulares que vamos a ocupar, de igual manera, se encuentran las palabras reservadas en este código entrada.

Asignamos las rutas donde va obtener codigo.txt y localizamos el out donde necesitamos que nos arroje el código generado. Léxico.java es el archivo generado por Jflex, y es el que vamos a utilizar para poder validar nuestras cadenas.

También tenemos a la clase Tokens.java, también generador por Jflex, donde podemos encontrar a las expresiones regulares que complementará el uso de Léxico.java. Esta clase es necesaria para poder entender lo que significa lo que el usuario escribió, dentro de las reglas de nuestro lenguaje.

Decidimos ocupar un código similar para la clase TestArchivo.java al visto en clase, el cual su pseudocódigo es el siguiente:

```
main(args) {  
  
    try  
  
        rd←abrirBufferLectura("Fuente.txt")    lexico←instanciarLexico(rd)  
        resultado←nulo  
  
        repeat  
  
            resultado←lexico.yylex()  
  
            if (resultado=nulo) then  
  
                print "(" + resultado + ")" print  
                lexico.lexema + "-> Lexema"  
  
            if (resultado=Tokens.ERROR) then  
  
                print "Línea " + lexico.getYyline() print  
                "Columna " + lexico.getYycolumn()  
  
            until (resultado=Tokens.ERROR or resultado=fin_archivo)  
  
        catch (e)  
  
            e.printStackTrace() }
```

Tomamos esta decisión porque este código nos permite una mejor visualización en los errores. Tanto en cadenas de caracteres inválidos (errores nuestros en la

ingresión de datos) o si estábamos teniendo una mala comprensión y análisis en nuestro código Léxico.java.

Conclusión

De acuerdo a los programas, logramos comprender mejor cómo se conforma la primera parte de un compilador, el analizador léxico.

Entendemos el funcionamiento de los lexemas, tokens y de las palabras reservadas dentro del analizador léxico. La manipulación e integración de estos elementos para poder generar un analizador léxico es crucial para un buen funcionamiento.

Analizamos las etapas en las que se fueron desarrollando las herramientas para la generación de estos analizadores léxicos. Comenzando con solo AFD, para proseguir con las tablas de transiciones y después utilizando las expresiones regulares para poder simplificar el trabajo.

También notamos como las expresiones regulares, prácticamente, simplifican la creación de Jflex. Entender el funcionamiento de este creador de analizadores léxicos, nos define puntualmente como es que el analizador léxico va a interactuar después con el analizador sintáctico, aunque, todavía no llegamos a esa parte.

El último programa es el que mejor ejemplifica a un analizador léxico, utilizando Jflex (nuestro programa fuente) para arrojar los tokens necesarios después de asignar los lexemas que vamos a ocupar, para desarrollar nuestro ejemplo con el txt y descubrir si la línea es válida o es inválida.

Bibliografía y referencias

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.

Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Educación.

Lesson: Basic I/O (The Java™ Tutorials > Essential Java Classes). (2024). Oracle.Com.

<https://docs.oracle.com/javase/tutorial/essential/io/index.html>

GeeksforGeeks. (2015, July 13). *Introduction of Lexical Analysis*. GeeksforGeeks.

<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Klein, G. (2023). *JFlex - JFlex The Fast Scanner Generator for Java*. Jflex.De.

<https://jflex.de/>

